

Laboratório de Concorrência · Aula 10

Rafael Blasques Morales · RA 241248722 · Turma A

Gerado em 09/10/2026, 21:11:20 · tempo registrado no cronômetro: 51:21

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

Na cooperação, uma pessoa precisa esperar uma outra pessoa fazer a ação, para executar a sua, como esperar alguém terminar de servir a comida para poder comer. Já competição, é aonde duas pessoas disputam um recurso, como duas pessoas disputando a vez de usar o único banheiro da casa

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

O esperado é imprimir 2.000.000

1ª execução (n = 1.000.000)

1009458

2ª execução (n = 1.000.000)

1450022

3ª execução (n = 1.000.000)

1935860

Saída do programa

não respondido



1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Não bateram, e também foram diferentes em cada execução. Isso acontece por conta da maneira que as threads se intercalam, uma gravação pode sobrescrever o incremento da outra

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

Basicamente, vamos supor que o contador vale 3. A lê 3 e B lê 3, cada um vai somar +1 em sua cópia e chega a 4. A grava 4 no contador, e B grava por cima (sobrescreve). É nesse momento que se perde

1.3 Com n = 1000, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Quando alteramos para apenas 1000, a chance do incremento se perder reduz muito, o problema é que o programa pode passar nesses testes menores, mas em produção, onde valores são podem ser maiores, pode quebrar.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

ABAABB → 6

AAABBB → 8

AABBAB → 6

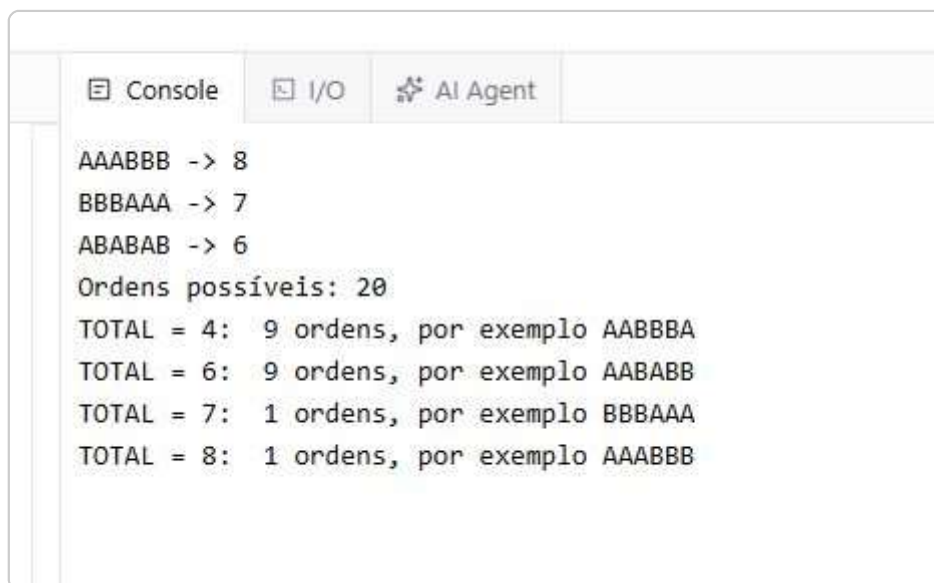
BBBAAA → 7

ABABAB → 6

ABBBA → 4

Saída do programa

não respondido



```
AAABBB -> 8
BBBAAA -> 7
ABABAB -> 6
Ordens possíveis: 20
TOTAL = 4: 9 ordens, por exemplo AABBBBA
TOTAL = 6: 9 ordens, por exemplo AABABB
TOTAL = 7: 1 ordens, por exemplo BBBAAA
TOTAL = 8: 1 ordens, por exemplo AAABBB
```

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

Existem 20 ordens possíveis e quatro resultados diferentes, porém apenas uma dá o resultado correto, que é 8

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Na ordem ABBABA, A lê 3, B lê 3, B calcula $3 \times 2 = 6$, A calcula $3 + 1 = 4$, B grava 6 e A grava 4 por último. A usou o valor antigo e sobrescreveu o resultado de B. Por isso a multiplicação de B se perdeu e o total ficou 4.

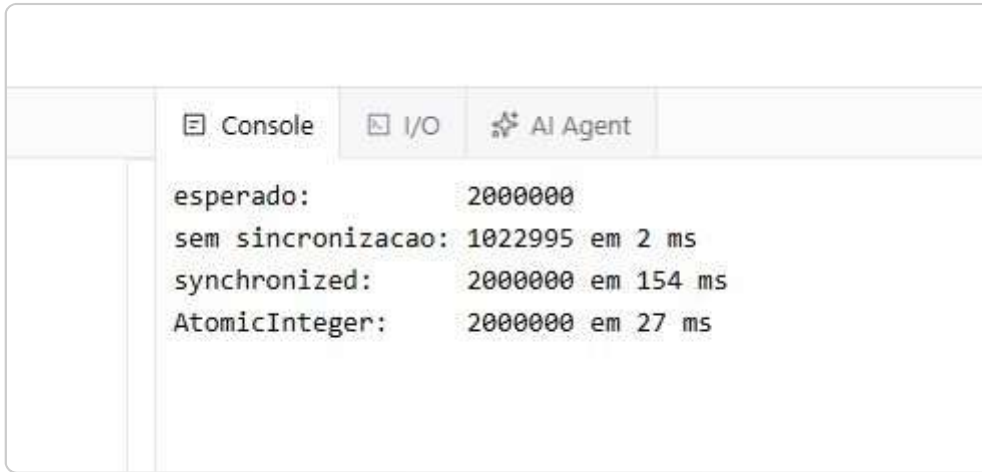
Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

A mais rápida será o semáforo, porém, como vimos acima, o resultado pode vir incorreto

Saída do programa

não respondido



```
esperado:          2000000
sem sincronizacao: 1022995 em 2 ms
synchronized:      2000000 em 154 ms
AtomicInteger:      2000000 em 27 ms
```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

Por monitor e operação atômica, o resultado saiu como esperado. Já o semáforo trouxe o valor incorreto. As versões corretas, tem uma proteção maior contra isso, mas com custo: Por monitor, o synchronized controla a entrada de uma thread por vez, e o atômico valida os incrementos

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não resolve, ele não transforma as duas alterações em uma operação única, precisa proteger a transferência inteira, não apenas o saldo

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

Não, por conta do deadlock não irão aparecer

Saídas (antes e depois da correção)

não respondido



```
IMPASSE: 2 threads presas
```

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

Porque A tinha X, mas precisava do Y, enquanto B tinha Y e esperava X. A única forma delas liberarem os recursos, seriam finalizando seu processo, porém precisavam da outra, então nunca vai acontecer

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

As duas pegam X, se alguma não conseguiu, ela vai esperar um processo liberar X para conseguir o recurso, sem ficar esperando com Y.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

Para 3, serão 200ms cada, 1 serão 600ms, e 6 100ms cada

O total é 600ms, já que são 6 tarefas de 100ms

Tempo com 1 trabalhador

600ms

Tempo com 3 trabalhadores

200ms

Tempo com 6 trabalhadores

100ms

Saída do programa

```
1 trabalhador(es):
trabalhador 1 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 1 fez a tarefa 3
trabalhador 1 fez a tarefa 4
trabalhador 1 fez a tarefa 5
trabalhador 1 fez a tarefa 6
1 trabalhadores, 6 tarefas de 100 ms: levou 600ms
```

```
3 trabalhador(es):
trabalhador 1 fez a tarefa 1
trabalhador 3 fez a tarefa 2
trabalhador 2 fez a tarefa 3
trabalhador 3 fez a tarefa 5
trabalhador 1 fez a tarefa 4
trabalhador 2 fez a tarefa 6
3 trabalhadores, 6 tarefas de 100 ms: levou 200ms
```

```
6 trabalhador(es):
trabalhador 3 fez a tarefa 6
trabalhador 2 fez a tarefa 4
trabalhador 1 fez a tarefa 1
trabalhador 5 fez a tarefa 3
trabalhador 4 fez a tarefa 2
trabalhador 6 fez a tarefa 5
6 trabalhadores, 6 tarefas de 100 ms: levou 100ms
```

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Com um trabalhador, as seis tarefas são feitas uma depois da outra, levando cerca de 600 ms. Com três, são aproximadamente duas rodadas de 100 ms. Com seis, as tarefas podem esperar seus 100 ms ao mesmo tempo. Os valores reais podem ter um pequeno atraso extra. O código não precisa de uma trava explícita para distribuir as tarefas porque o canal coordena os envios e recebimentos, e cada trabalhador usa sua própria variável de tarefa. O canal e o WaitGroup já fornecem sincronização. Esse ganho se deve ao trabalho simulado com espera; não garante o mesmo ganho para cálculos pesados.

Saídas (antes e depois da troca)*não respondido*

```

fatal error: all goroutines are asleep - deadlock!

goroutine 1 [chan send]:
main.main()
    /box/main.go:9 +0x38
exit status 2

```

```

1

```

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

Sem buffer, o envio só termina quando alguém recebe a mensagem. Só que a única goroutine fica parada no envio e nunca chega à linha que receberia. O runtime detecta o deadlock. Com espaço para uma mensagem, o envio guarda o valor no canal e continua. A próxima linha consegue receber esse valor e imprimir 1.

Prática 6: O compilador como guarda**Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?**

Vai dar erro, como dito acima, o RUST não permite as threads alterarem a mesma variável

Saídas (o erro e o resultado com move)

```

println!("{}", contador);
}
Exit 1
error[E0373]: closure may outlive the current function, but it borrows `contador`, which is
owned by the current function
  → HelloWorld.rs:7:27
   |
7 |     let t = thread::spawn(|| {
   |                               ^^ may outlive borrowed value `contador`
8 |         contador += 1;           // a thread altera a variável de main
   |         ----- `contador` is borrowed here
   |
note: function requires argument type to outlive `'static`
  → HelloWorld.rs:7:13
   |
7 |     let t = thread::spawn(|| {
   |     ^-----^
8 | |         contador += 1;           // a thread altera a variável de main
9 | |     });
   | |_____^

```

help: to force the closure to take ownership of `contador` (and any other referenced variables), use the `move` keyword

```
|
7 |     let t = thread::spawn(move || {
|                               +++++
```

error[E0502]: cannot borrow `contador` as immutable because it is also borrowed as mutable
→ HelloWorld.rs:11:20

```
|
7 |         let t = thread::spawn(|| {
|             -               -- mutable borrow occurs here
|             -----|
|             |
8 |             |         contador += 1;                // a thread altera a variável de main
|             |         ----- first borrow occurs due to use of `contador` in closure
9 |             |     });
|             |----- argument requires that `contador` is borrowed for `static`
10 |         t.join().unwrap();
11 |         println!("{}", contador);
|             ^^^^^^^^^ immutable borrow occurs here
|
```

note: requirement that the value outlives `static` introduced here

→
/rustc/48a229ceaefd4985c50990b14116b6d856af0985/library/std/src/thread/functions.rs:128:14

error: aborting due to 2 previous errors

Some errors have detailed explanations: E0373, E0502.

For more information about an error, try `rustc --explain E0373`.

// Prática 6a - Rust: tentando compartilhar uma variável entre threads
// Cole em <https://onecompiler.com/rust>
use std::thread;

```
fn main() {
    let mut contador = 0;
    let t = thread::spawn(move || {
        contador += 1;                // a thread altera a variável de main
    });
    t.join().unwrap();
}
```

262 ms

warning: value captured by `contador` is never read

```
→ HelloWorld.rs:8:9
|
8 |         contador += 1;                // a thread altera a variável de main
|         ^^^^^^^^^
|
= help: did you mean to capture by reference instead?
= note: `#[warn(unused_assignments)]` (part of `#[warn(unused)]`) on by default
```

warning: value assigned to `contador` is never read

```
→ HelloWorld.rs:8:9
|
8 |         contador += 1;                // a thread altera a variável de main
|         ^^^^^^^^^^^^^^^^^^^^^
```

```
|  
= help: maybe it is overwritten before being read?
```

```
warning: 2 warnings emitted
```

```
0
```

Saída do programa

```
contador = 400000
```

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

Nesse caso, o número inteiro implementa Copy. Com move, a thread recebe uma cópia do contador e soma 1 nessa cópia. O contador que ficou em main continua valendo 0. Esperar a thread com join não copia o valor de volta.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java, a corrida da Prática 1 aparece durante a execução, quando o resultado sai errado, e pode até passar despercebida. No exemplo em Rust, o empréstimo inadequado é recusado na compilação. Isso ajuda a impedir corridas de dados no código seguro antes de colocar o programa para rodar. O custo é aprender e respeitar as regras de posse, empréstimo e compartilhamento

Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

A - D - B - C

Saída do programa

```
Console  
A  
D  
C  
B  
E: timer de 10 ms rodou depois de 300 ms
```

7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

ADCB saiu nessa ordem. A e D não tem timer, então executa linha a linha, C executa em sequência por causa da microtarefa promisse, e o callback de timeout imprime B

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

O laço ocupado segura a thread por cerca de 300 ms. O timer fica pronto antes, mas seu callback só pode rodar quando essa thread estiver livre. Em um servidor Node, o calculo pesado pode atrasar vários usuários

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

Pode acontecer uma condição de corrida: as duas compras leem que ainda existe um lugar e ambas tentam vendê-lo. Eu faria a verificação e a reserva em uma única operação atômica no banco de dados. Assim, apenas uma pessoa consegue o último ingresso, mesmo que os pedidos cheguem juntos.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi que uma operação tão simples quanto somar 1 pode dar errado quando duas threads mexem no mesmo valor sem proteção.